

React — Day 2

State, Forms & Lifting State Up

The most important session of the course

Prerequisites: Day 1 — components, JSX, props, lists and keys

Environment: Node.js 18+, VS Code, Chrome, React DevTools

1. Recap and the rules of hooks

Day 1 covered the "UI" half of *UI is a function of state*. Today is the "state" half.

Everything so far has been static. A component received props and returned JSX, and the only way anything changed was if you edited the code. Today components gain memory.

Where we left off

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

`useState` returns a pair: the current value, and a function that changes it and tells React to re-render. The names are yours to choose — `[count, setCount]` is convention, not requirement.

A component can hold as many state variables as it needs

```
const [name, setName] = useState("");
const [age, setAge] = useState(0);
const [tags, setTags] = useState([]);
const [user, setUser] = useState(null);
```

Group values into one object when they change together; keep them separate when they change independently. Both are correct — this is a judgement call, not a rule.

The rules of hooks

Two rules, and they are not negotiable:

1. Only call hooks at the **top level** of a component. Never inside a condition, loop, or nested function.
2. Only call hooks from React function components or from your own custom hooks.

```
// Broken – the hook is inside a condition
function Profile({ isLoggedIn }) {
  if (isLoggedIn) {
    const [name, setName] = useState("");
  }
}

// Correct – always called, condition applied elsewhere
function Profile({ isLoggedIn }) {
  const [name, setName] = useState("");
  if (!isLoggedIn) return <p>Please log in</p>;
}
```

Why the rule exists — this is worth understanding, not just obeying.

React does not know your variable names. It stores state in an ordered list per component and matches the first `useState` call to slot 0, the second to slot 1, and so on. Order is the only identity it has. Put a hook inside an `if` and the slots shift between renders, so React hands state belonging to one variable to a completely different one. The rule is not arbitrary bureaucracy; it is the direct consequence of how hooks are implemented.

Lazy initial state

```
useState(expensiveCalculation()) // runs on EVERY render, result discarded
useState(() => expensiveCalculation()) // runs once, on first render only
```

The initial value is only used the first time. If computing it is costly, pass a function instead of a value so React can call it only when it is actually needed.

2. State is a snapshot

This section is the conceptual heart of the day. If something here feels shaky, stop and ask — the rest of the course assumes it.

Predict the result before reading on

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

Most people say the count goes up by 3. It goes up by **1**.

Why

`count` is not a live variable that other code can read the latest value from. It is a `const`, captured by *this particular render*. During this render its value is `0` and it will never be anything else. So the three lines are not three increments — they are the same instruction written three times:

```
setCount(0 + 1); // set it to 1
setCount(0 + 1); // set it to 1
setCount(0 + 1); // set it to 1
```

React then **batches** these: rather than re-rendering after each call, it collects all state updates from the event handler and performs one render at the end. The final instruction wins, and the count becomes 1.

The mental model.

Each render is a photograph. The values inside it are frozen at the moment it was taken. Calling a setter does not edit the photograph you are standing in — it schedules a new one to be taken.

The proof

Run this and watch what the alert says:

```
function handleClick() {
  setCount(count + 1);
  setTimeout(() => alert(count), 2000); // alerts the OLD value
}
```

Two seconds later the screen shows the new count, but the alert shows the old one. The arrow function captured `count` from the render it was created in, and that render's `count` never changed. Nothing is broken — this is exactly what a snapshot means.

The two forms of a setter

Form	Meaning
<code>setCount(count + 1)</code>	"Set it to this specific value." Uses the value from the current render.
<code>setCount(c => c + 1)</code>	"Update it from whatever it currently is." React passes in the latest pending value.

The updater function fixes the original bug:

```
setCount(c => c + 1);  
setCount(c => c + 1);  
setCount(c => c + 1);    // now the count goes up by 3
```

React queues the three functions and runs them in order: 0 becomes 1, 1 becomes 2, 2 becomes 3.

When to use which.

Use the updater form whenever the new value depends on the old one — increments, toggles, appending to a list. Use the direct form when you are setting a value that does not depend on the previous one, such as assigning what the user typed. When in doubt the updater form is always safe.

The render cycle

Keep this sequence in mind for the rest of the course:

1. Something happens — a click, a keystroke, a timer.
2. A setter is called, which tells React this component's state has changed.
3. React re-runs the component function from the top. All your `const` declarations execute again.
4. The function returns fresh JSX describing the new screen.
5. React compares it against the previous JSX and works out the minimum DOM changes.
6. React applies only those changes.

Note step 3: the entire function body runs again on every render. This is why putting a `console.log` at the top of a component is such an effective debugging technique — it fires once per render, so you can see exactly how often and when your component is running.

3. Never mutate state

The bug that produces no error

```
const [items, setItems] = useState(["apples", "bread"]);

function addItem() {
  items.push("cheese"); // the array really does change
  setItems(items);      // ...but the screen does not update
}
```

The array now contains three entries. The screen still shows two. No warning appears in the console.

Why

When you call a setter, React compares the new value against the old one using `Object.is` — a reference comparison for objects and arrays. You handed it *the same array* you were already holding, so React concludes nothing changed and skips the render entirely.

This is the worst category of bug:

your data is correct and your screen is stale, with nothing to tell you something went wrong. The fix is to always produce a **new** array or object rather than editing the existing one.

```
function addItem() {
  setItems([...items, "cheese"]); // new array – React sees the change
}
```

The patterns to memorise

Operation	Never	Always
Add to the end	<code>arr.push(x)</code>	<code>[...arr, x]</code>
Add to the start	<code>arr.unshift(x)</code>	<code>[x, ...arr]</code>
Remove an item	<code>arr.splice(i, 1)</code>	<code>arr.filter(a => a.id !== id)</code>
Update one item	<code>arr[i].done = true</code>	<code>arr.map(a => a.id === id ? { ...a, done: true } : a)</code>
Insert in the middle	<code>arr.splice(i, 0, x)</code>	<code>[...arr.slice(0, i), x, ...arr.slice(i)]</code>
Sort	<code>arr.sort()</code>	<code>[...arr].sort()</code>
Reverse	<code>arr.reverse()</code>	<code>[...arr].reverse()</code>
Change a field	<code>obj.name = "x"</code>	<code>{ ...obj, name: "x" }</code>

The two that catch everyone.

`sort()` and `reverse()` modify the array in place and return it. They look like they produce a new array, and they do not. Always copy first with the spread operator.

Spread is shallow

Spreading copies only the top level. Nested objects are still shared references:

```
const [user, setUser] = useState({
  name: "Sara",
  address: { city: "Alexandria", zip: "21500" }
});

// Broken – address is still the same object
setUser({ ...user, address: { ...user.address } }); // this part is fine
user.address.city = "Cairo";                       // this mutates state

// Correct – spread at every level you are changing
setUser({
  ...user,
  address: { ...user.address, city: "Cairo" }
});
```

Deeply nested state is a signal that the state shape is wrong. Flatten it where you can — it is almost always easier than writing four levels of spread.

A useful safety net

React's StrictMode, which Vite enables by default, helps catch some of these mistakes during development by running certain code twice. If your component behaves oddly only in development, that is usually StrictMode surfacing a real bug rather than causing one.

4. Forms and controlled components

The controlled input pattern

```
function NameForm() {
  const [name, setName] = useState("");

  return (
    <>
      <input value={name} onChange={e => setName(e.target.value)} />
      <p>Hello {name || "stranger"}</p>
    </>
  );
}
```

Two halves make it work. `value={name}` means the input displays whatever is in state. `onChange` means every keystroke writes back to state. Remove either half and the input stops working — remove `onChange` and it freezes; remove `value` and React loses track of it.

Why bother?

Because state becomes the single source of truth. You can validate as the user types, disable the submit button until the form is valid, transform input (force uppercase, strip spaces), or reset the form by setting state back to empty. None of that is convenient when the value lives only in the DOM.

One handler for an entire form

A separate handler per field does not scale. Use the `name` attribute and a computed key:

```
const [form, setForm] = useState({
  name: "", email: "", subscribe: false
});

function handleChange(e) {
  const { name, value, type, checked } = e.target;
  setForm(prev => ({
    ...prev,
    [name]: type === "checkbox" ? checked : value
  }));
}

<input name="name" value={form.name} onChange={handleChange} />
<input name="email" value={form.email} onChange={handleChange} />
<input name="subscribe" type="checkbox"
  checked={form.subscribe} onChange={handleChange} />
```

The square brackets in `[name]` are a computed property key — they use the *value* of the variable `name` as the key, rather than the literal string "name". This one line lets a single handler serve any number of fields.

Input types and their quirks

Element	What controls it
text, email, number	<code>value</code> plus <code>onChange</code>
checkbox	<code>checked</code> , not <code>value</code> . Read <code>e.target.checked</code> .
radio	<code>checked={form.choice === "a"}</code> on each option
<code><textarea></code>	Takes a <code>value</code> prop. Unlike HTML, the text does not go between the tags.
<code><select></code>	<code>value</code> goes on the select. Do not use <code>selected</code> on options.
number input	<code>e.target.value</code> is always a string . Convert with <code>Number()</code> .

"A component is changing an uncontrolled input to be controlled."

This warning means `value` was `undefined` on the first render and became a string later. Initialise text state to `""` rather than `undefined` or `null`.

Submitting

```
function handleSubmit(e) {
  e.preventDefault();          // stop the browser reloading the page
  console.log(form);
  setForm({ name: "", email: "", subscribe: false }); // reset
}

<form onSubmit={handleSubmit}>
  ...
  <button type="submit">Send</button>
</form>
```

Put the handler on the `<form>` rather than the button, so that pressing Enter in a field also submits. Without `e.preventDefault()` the browser performs a full page reload and your app restarts.

5. Lifting state up

This is the skill that separates people who can build applications from people who can only build components.

The problem

A `SearchBar` owns the query the user typed. A `ResultsList` needs that query to filter what it displays. They are siblings, and siblings cannot see each other's state.

```
function App() {
  return (
    <>
      <SearchBar />      {/* owns "query" */}
      <ResultsList />    {/* needs "query" – but cannot reach it */}
    </>
  );
}
```

The solution

Move the state up to the nearest component that contains both, then pass it back down.

```
function App() {
  const [query, setQuery] = useState("");

  return (
    <>
      <SearchBar query={query} onQueryChange={setQuery} />
      <ResultsList query={query} />
    </>
  );
}

function SearchBar({ query, onQueryChange }) {
  return <input value={query} onChange={e => onQueryChange(e.target.value)} />;
}
```

The procedure

1. Identify every component that needs to read or write the value.
2. Find their nearest common parent.
3. Move the `useState` into that parent.
4. Pass the value down as a prop.
5. Pass a setter or a handler function down as a prop.

Name the pattern: data flows down, events flow up.

A parent gives a child data through props. A child tells the parent that something happened by calling a function the parent supplied. The child never reaches upward on its own. This is why React apps stay predictable as they grow — when a value is wrong, there is exactly one place that owns it.

Controlled and uncontrolled components

The same distinction you met with inputs applies to your own components:

Type	Meaning
Uncontrolled	The component owns its state internally. Simpler to use, but the parent cannot read or change it.
Controlled	The parent owns the state and passes it in. More wiring, but the parent is in charge.

Start uncontrolled. Lift the state up only when something else genuinely needs it — lifting early adds props that nothing benefits from.

Where should state live?

As low in the tree as possible, but high enough that everything needing it can reach it. State that has been lifted higher than necessary causes unrelated components to re-render and makes the parent harder to read.

6. Derived state

A short section that prevents more bugs than anything else today.

The wrong way

```
const [items, setItems]      = useState([]);
const [filtered, setFiltered] = useState([]); // a second source of truth
const [total, setTotal]      = useState(0);   // a third
```

Now every function that changes `items` must remember to update `filtered` and `total` as well. Miss one path — a delete, a bulk import, an undo — and the numbers silently disagree with the list. This is the single most common cause of "the count is wrong" bugs.

The right way

```
const [items, setItems] = useState([]);
const [filter, setFilter] = useState("all");

// Calculated fresh on every render – no second source of truth
const filtered = items.filter(i => filter === "all" || i.status === filter);
const total    = items.reduce((sum, i) => sum + i.price, 0);
const remaining = items.filter(i => !i.done).length;
```

These are ordinary `const` declarations inside the component body. They recalculate every render, which is exactly what you want: they cannot go stale, because they are recomputed from the source of truth every single time.

The rule: if you can calculate it from existing state, do not store it.

Ask of every piece of state you are about to create — could I work this out from something I already have? If yes, calculate it instead.

"Isn't recalculating on every render slow?"

Almost never. Filtering a few hundred items takes a fraction of a millisecond, and React re-renders far less often than people assume. Correctness first. There are tools for optimising genuinely expensive calculations, and they are covered later in the course — but reaching for them before you have measured a problem creates complexity for no benefit.

What state should actually hold

Store in state	Calculate during render
The list of items	The filtered or sorted view of it
The active filter name	How many items match it
The raw form field values	Whether the form is valid
Which row id is selected	The full object for that row
The cart contents	The subtotal, tax, and grand total

7. useEffect

Every hook so far has kept a component self-contained. `useEffect` is the doorway out — it lets a component synchronise with something that is not React.

useEffect is for synchronising with an external system.

"External" means anything React does not control: the network, browser APIs like the document title or local storage, timers, subscriptions, and third-party libraries.

Shape

```
useEffect(() => {  
  // the effect – runs AFTER React has updated the DOM  
  
  return () => {  
    // the cleanup – runs before the next effect, and on unmount  
  };  
}, [dependencies]);
```

Effects run *after* the browser has painted, not during render. That is deliberate: it keeps rendering fast and pure.

The dependency array decides when it runs

Written as	Runs
<code>useEffect(fn)</code>	After every render. Rarely what you want.
<code>useEffect(fn, [])</code>	Once, after the first render only.
<code>useEffect(fn, [a, b])</code>	After the first render, and again whenever <code>a</code> or <code>b</code> changes.

```
// Keep the browser tab title in sync with the count  
useEffect(() => {  
  document.title = `${count} items`;  
}, [count]);
```

The document title is not React's — it belongs to the browser. That is what makes this a legitimate effect.

Cleanup

Anything an effect starts, it must be able to stop. Without cleanup you leak timers, listeners, and connections every time the component re-runs or unmounts.

```
useEffect(() => {
  const id = setInterval(() => setTick(t => t + 1), 1000);

  return () => clearInterval(id); // without this, a new timer
}, []);                          // every render, none stopped
```

Your effect will run twice in development, and that is intentional.

StrictMode deliberately mounts, unmounts, and remounts each component once. If a double run causes a visible problem, the effect is missing its cleanup — StrictMode is exposing a real bug rather than creating one. This does not happen in production builds.

When NOT to use it

This matters more than the syntax. Most bad React code is an effect doing a job that belonged somewhere else.

What you want	The right tool
A value computed from state or props	Calculate it during render — see section 6. Not an effect.
Something to happen when a user clicks	The event handler. Not an effect.
To reset state when a prop changes	A <code>key</code> on the component.
To fetch data when the page opens	An effect — this one genuinely qualifies.
To subscribe, or start a timer	An effect, with cleanup.

```
// WRONG – an effect keeping derived state in sync
const [total, setTotal] = useState(0);
useEffect(() => {
  setTotal(items.reduce((s, i) => s + i.price, 0));
}, [items]);

// RIGHT – just calculate it
const total = items.reduce((s, i) => s + i.price, 0);
```

The wrong version renders twice for every change, can briefly show a stale total, and adds a second source of truth. The right version cannot go out of date because it is recomputed every render.

Fetching data

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  let ignore = false; // guards against a race
  setLoading(true);

  fetch(`/api/products?q=${query}`)
    .then(res => res.json())
    .then(json => { if (!ignore) setData(json); })
    .catch(err => { if (!ignore) setError(err); })
    .finally(() => { if (!ignore) setLoading(false); });

  return () => { ignore = true; }; // cleanup
}, [query]);
```

The `ignore` flag solves a real problem. If the query changes quickly, several requests are in flight at once and they can come back out of order — an older, slower response would otherwise overwrite a newer one. Cleanup marks the stale request as unwanted before the next effect starts.

Two traps

The infinite loop

```
useEffect(() => {
  setCount(count + 1); // sets state...
}, [count]);           // ...which changes the dependency...
                        // ...which runs the effect again. Forever.
```

Objects and arrays as dependencies

```
// Broken – a NEW object every render, so never equal to last time
useEffect(() => { ... }, [{ id: userId }]);

// Fine – a primitive, compared by value
useEffect(() => { ... }, [userId]);
```

React compares dependencies with `Object.is`. Two objects with identical contents are still two different objects, so the effect fires on every render.

Before writing an effect, ask what external system it synchronises with.

If you cannot name one, you probably do not need an effect. That single question prevents most of the misuse.

8. Fetching data

React has no fetching feature. There is no `<Fetch>` component and no built-in data layer — you use the browser's `fetch` exactly as you would without React. That leaves only two React-specific questions:

1. **Where does the result go?** Into state, because the UI must update when it arrives.
2. **Where does the call go?** Into an effect, because it is a side effect and render must stay pure.

Every fetch has three possible states: loading, error, and data.

Beginners write for the third one only, then wonder why the page flashes "nothing found" for a second, or shows a blank screen forever when the API is down. Design for all three from the start and most of the bugs never happen.

Step 1 — the attempt that breaks

```
function ProductList() {
  const [products, setProducts] = useState([]);

  fetch("/api/products")           // runs on EVERY render
    .then(res => res.json())
    .then(setProducts);           // which causes another render

  return <ul>...</ul>;
}
```

Open the Network panel and it fills with hundreds of identical requests. The fetch resolves, state is set, the component re-renders, the function body runs again, another fetch goes out. Forever. Render describes the screen; it does not make things happen.

Step 2 — move it into an effect

```
useEffect(() => {
  fetch("/api/products")
    .then(res => res.json())
    .then(setProducts);
}, []);                                // [] = once, after the first render
```

One request now. But this version still lies: while the request is in flight `products` is `[]`, so the page renders an empty list as though the API had returned nothing. If the request fails it stays empty forever with no explanation.

Step 3 — all three states

```
const [products, setProducts] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  setLoading(true);
  setError(null); // clear the previous error

  fetch("/api/products")
    .then(res => {
      if (!res.ok) throw new Error(`Request failed: ${res.status}`);
      return res.json();
    })
    .then(data => setProducts(data))
    .catch(err => setError(err.message))
    .finally(() => setLoading(false));
}, []);

if (loading) return <p>Loading...</p>;
if (error) return <p className="error">{error}</p>;
return <ul>...</ul>;
```

The `res.ok` check is not optional.

`fetch` only rejects on a *network* failure — no connection, DNS error, CORS block. A 404 or a 500 is a **successful** response as far as `fetch` is concerned. Without this check your `.catch` never fires, and you try to render an error page as if it were your data.

Note the early returns. `if (loading) return ...` reads far better than wrapping the whole component in nested ternaries.

Step 4 — when the URL depends on state

```
useEffect(() => {
  let ignore = false;

  fetch(`/api/search?q=${query}`)
    .then(res => res.json())
    .then(data => { if (!ignore) setResults(data); });

  return () => { ignore = true; };
}, [query]);
```

Type "lap" quickly and three requests are in flight at once. They do not come back in the order they were sent — the slow response for "l" can land *after* the fast response for "lap" and overwrite it, so the user sees results for a query they finished typing seconds ago.

React runs the cleanup of the previous effect before starting the next one. That flips `ignore` on the stale request, so when it finally returns its result is discarded.

Step 5 — the async / await version

```
// Does NOT work – React warns
useEffect(async () => { ... }, []);
```

The effect callback must return a cleanup function or nothing, and an `async` function always returns a Promise. Declare the async function inside and call it:

```
useEffect(() => {
  let ignore = false;

  async function load() {
    try {
      setLoading(true);
      const res = await fetch(url);
      if (!res.ok) throw new Error(`Request failed: ${res.status}`);
      const data = await res.json();
      if (!ignore) setProducts(data);
    } catch (err) { if (!ignore) setError(err.message); }
    finally { if (!ignore) setLoading(false); }
  }

  load();
  return () => { ignore = true; };
}, [url]);
```

Identical behaviour to step 3. Pick whichever reads more clearly to you.

Step 6 — extract a custom hook

Steps 3 and 5 are the same twenty lines. A custom hook is simply a function starting with `use` that calls other hooks — this is how logic gets shared between components.

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => { /* exactly the code from step 3 */ }, [url]);

  return { data, loading, error };
}

// every component that needs data is now one line
const { data, loading, error } = useFetch("/api/products");
```

The mistakes to watch for

Mistake	What happens
Fetching during render	Infinite request loop
No <code>res.ok</code> check	404s and 500s silently treated as success
No loading state	Empty-state flash, or a blank screen
No error state	Failures look identical to "no results"
Missing cleanup	Stale responses overwrite fresh ones
<code>useEffect(async () => ...)</code>	React warning; cleanup cannot work
Missing dependency	The fetch never re-runs when the query changes
Object in the dependency array	Refetches on every single render

The other ways, and where this goes

Almost nobody hand-writes the code above in production. It has no caching, so leaving a page and returning refetches everything. Two components asking for the same data fire two requests. There is no retry and no background refresh.

Approach	When it is used
<code>fetch</code> in <code>useEffect</code>	What you have just learned. Fine for small apps and for understanding the problem.
A custom <code>useFetch</code> hook	The same thing, deduplicated. The last step before a library.
TanStack Query	The standard answer in production. Caching, deduplication, retries and race handling are all built in.
<code>use()</code> with Suspense	React 19. The component suspends until the promise resolves; loading state moves to a <code><Suspense></code> boundary.
Server Components	The fetch happens on the server before any HTML is sent. No effect, no loading state, no client JavaScript.

```
// TanStack Query – the whole of steps 1 to 6, collapsed
const { data, isLoading, error } = useQuery({
  queryKey: ["products", query],
  queryFn: () => fetch(`/api/search?q=${query}`).then(r => r.json()),
});
```

So why learn the manual version?

Because a library's API only makes sense once you have felt the problems it solves.

`queryKey` looks arbitrary until you have hand-managed a dependency array, and

`isLoading` looks like boilerplate until you have forgotten a loading state and watched the empty-state flash.

9. Spot the bug

Each snippet is broken in exactly one way, and all of them come from today's material.

1. The counter only ever goes up by one

```
function handleClick() {  
  setCount(count + 1);  
  setCount(count + 1);  
}
```

2. The new item never appears on screen

```
function addItem(item) {  
  items.push(item);  
  setItems(items);  
}
```

3. The list disappears entirely after sorting

```
function sortItems() {  
  setItems(items.sort((a, b) => a.price - b.price));  
}
```

4. Console warning: changing an uncontrolled input to controlled

```
const [email, setEmail] = useState();  
  
return <input value={email} onChange={e => setEmail(e.target.value)} />;
```

5. The page reloads and the form empties whenever it is submitted

```
function handleSubmit(e) {  
  console.log(form);  
}  
  
<form onSubmit={handleSubmit}>...</form>
```

6. The total is sometimes wrong after deleting an item

```
const [items, setItems] = useState([]);  
const [total, setTotal] = useState(0);  
  
function remove(id) {  
  setItems(items.filter(i => i.id !== id));  
}
```

7. Typing "5" into the number field and adding 1 produces "51"

```
<input type="number" value={qty}  
      onChange={e => setQty(e.target.value)} />  
  
<p>One more would be {qty + 1}</p>
```

Answers

are in the error table in the quick reference at the back. Try all seven first.

Task

Build a product list that loads from a real API, then let the user act on it. This uses everything from today: fetching in an effect, the three request states, immutable updates, a controlled input, and derived values.

The API

<https://dummyjson.com/products> is a free fake REST API. No key, no signup, no backend to run.

```
fetch("https://dummyjson.com/products?limit=12")
  .then(res => res.json())
  .then(data => console.log(data));

// The response is an OBJECT, not an array:
// { products: [ ... ], total: 194, skip: 0, limit: 12 }
//
// So the list you want is data.products – a very common first mistake.
//
// Each product has: id, title, price, category, rating, stock, thumbnail
```

Tip for seeing the loading state.

The API is fast enough that "Loading..." can flash by unnoticed. Add `&delay=1000` to the URL to slow the response down to a second while you build.

Requirements

1. Fetch twelve products when the page loads, and handle all three states — **loading**, **error**, and **data**. Do not skip the error state; you can test it by breaking the URL on purpose.
2. Show each product as a card with its title, price, and category.
3. Give each card a **Delete** button that removes it from the list. Remember that state must never be mutated.
4. Add a **Favourite** button on each card that toggles a star on and off. Toggling one card must not affect the others.
5. Add a search input that filters the visible products by title as the user types. The filtered list must be **calculated during render**, never stored in state.
6. Show a line reading "8 of 12 products" that reflects the current filter and updates as items are deleted.

The parts worth thinking about

Requirement	What it is testing
Fetching on load	<code>useEffect</code> with an empty dependency array, and the three states
Delete	Immutable removal with <code>filter</code>
Favourite	Immutable update with <code>map</code> and object spread
Search	A controlled input, and filtering during render rather than in state
The count	Derived values — nothing extra stored

The search is not a dependency here.

You already have every product in state, so filtering is ordinary JavaScript that belongs in render. Do not add `search` to a dependency array and refetch on every keystroke — the network is not involved. A search term only becomes a dependency when the *server* does the filtering, as in section 8.

Quick reference

Errors you will see today

Message or symptom	What it actually means
The counter only increments once	Multiple setter calls used the same snapshot. Use <code>setCount(c => c + 1)</code> .
State changed but nothing re-rendered	You mutated instead of replacing. Spread into a new array or object.
The list vanishes or behaves oddly after sorting	<code>sort()</code> mutates in place. Use <code>[...items].sort()</code> .
Changing an uncontrolled input to be controlled	<code>useState()</code> with no argument gives <code>undefined</code> . Initialise to <code>""</code> .
The page reloads on form submit	Missing <code>e.preventDefault()</code> in the submit handler.
A derived number disagrees with the list	You stored something you should calculate. Remove the extra state.
Numbers concatenate instead of adding	<code>e.target.value</code> is always a string. Wrap it in <code>Number()</code> .
Rendered more hooks than during the previous render	A hook is inside a condition or loop. Move it to the top level.
A 404 or 500 was treated as success	You skipped <code>if (!res.ok) throw</code> . <code>fetch</code> only rejects on network failure.
Search shows results for an old query	Missing cleanup. Use the <code>ignore</code> flag pattern.
The effect loops forever	The effect sets state that is also in its dependency array.
The effect runs twice on load	StrictMode in development. If it breaks things, the effect needs cleanup.
Too many re-renders	You called a setter during render instead of inside a handler. Use <code>onClick={() => setX(1)}</code> , not <code>onClick={setX(1)}</code> .

Syntax at a glance

Pattern	Syntax
Declare state	<code>const [v, setV] = useState(initial);</code>
Lazy initial value	<code>useState(() => expensive())</code>
Set directly	<code>setV(newValue)</code>
Set from previous	<code>setV(prev => prev + 1)</code>
Add to array	<code>setArr([...arr, item])</code>
Remove from array	<code>setArr(arr.filter(a => a.id !== id))</code>
Update one item	<code>setArr(arr.map(a => a.id === id ? {...a, done: true} : a))</code>
Sort safely	<code>setArr([...arr].sort(fn))</code>
Update object field	<code>setObj({ ...obj, name: "x" })</code>
Update nested field	<code>setObj({ ...obj, a: { ...obj.a, b: 1 } })</code>
Controlled text input	<code>value={v} onChange={e => setV(e.target.value)}</code>
Controlled checkbox	<code>checked={v} onChange={e => setV(e.target.checked)}</code>
Generic field handler	<code>setForm(p => ({ ...p, [e.target.name]: e.target.value }))</code>
Form submit	<code>onSubmit={e => { e.preventDefault(); ... }}</code>
Derived value	<code>const total = items.reduce((s, i) => s + i.price, 0);</code>
Pass a handler down	<code><Child onChange={setV} /></code>
Effect on mount only	<code>useEffect(() => { ... }, [])</code>
Effect when a value changes	<code>useEffect(() => { ... }, [value])</code>
Effect with cleanup	<code>useEffect(() => { ...; return () => stop(); }, [])</code>
Check an HTTP error	<code>if (!res.ok) throw new Error(res.status)</code>
Guard a stale response	<code>let ignore = false; ... return () => { ignore = true; }</code>